Ziira

Ziira Core 1.0 Technical Specification

Architecture, data model, engines and build plan for the engineering team.

SIMPLIFY. CONNECT. GROW.

Version 1.0 · October 2026

Confidential

CONTENTS

Ziira Core 1.0 Technical Specification

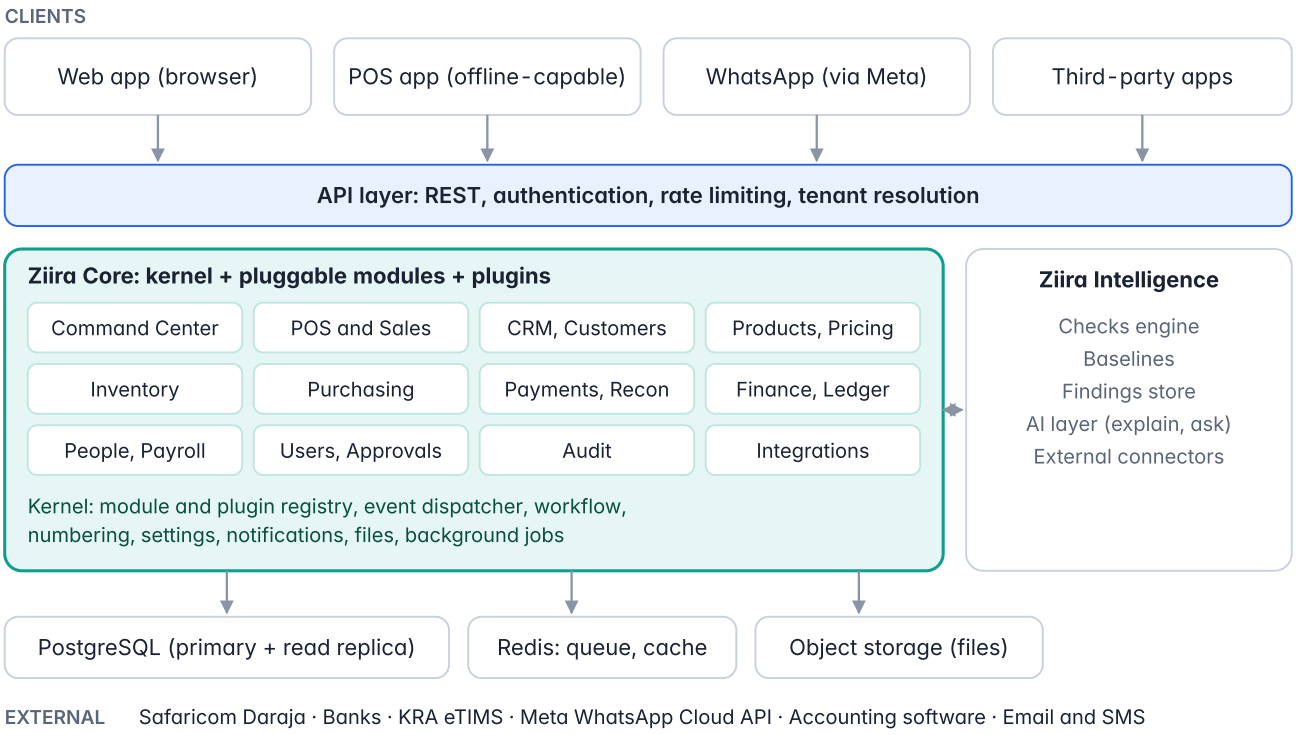
This document describes how Ziira Core 1.0 is built and how each part works. It is written for the engineering team and complements the *Ziira Core 1.0* and *Ziira Intelligence* product documents.

01	Architecture	Components and technology stack
02	Module and plugin system	Kernel, modules, plugins, lifecycle
03	Code structure and standards	Layout, layers, coding rules
04	Tenancy and organisation model	Tenants, companies, branches, locations
05	Data conventions	Rules every table and document follows
06	Modules	Entities, workflows and events for all 12
07	Accounting engine	Double entry, posting rules, periods
08	Inventory engine	Stock ledger, costing, batches
09	Point of sale and offline operation	Devices, sync, till sessions
10	Payments and reconciliation	M-Pesa, banks, matching engine
11	eTIMS integration	Tax invoicing with KRA
12	Payroll engine	Calculation, rate tables, outputs
13	Access control and workflow	Roles, scopes, approvals, rules
14	Audit trail	What is recorded and how
15	API, events and integrations	REST, webhooks, imports, exports
16	Ziira Intelligence interface	Checks, findings, AI layer, actions
17	WhatsApp channel	Messaging, identity, approvals
18	Security and data protection	Authentication, encryption, compliance
19	Infrastructure and operations	Deployment, backups, monitoring
20	Testing and quality	How correctness is proven
21	Build plan	Order of work and exit criteria
22	Open decisions	Items to settle before or during build
A	Database schema	All 11 schemas and 110 tables

01 · ARCHITECTURE

Architecture

Ziira Core is a multi-tenant web application built as a small kernel with pluggable modules and plugins: one deployable backend, one primary database, an event bus, and separate clients for web, point of sale and WhatsApp.



Why one backend with pluggable modules

- The modules share one ledger and one stock ledger, and most operations touch several modules in one database transaction (a sale updates stock, payments and the books together). A monolith keeps these atomic.
- One deployment is simpler to build, test and operate for a small team.
- Modules communicate only through public interfaces and events, never by reading each other's tables, so any module can be enabled, disabled or replaced without changing the others (section 02).

Technology stack

Layer	Choice	Purpose
Database	PostgreSQL 16+	Transactions, row-level security, JSON, reporting
Language	Python 3.12+ with full type hints	Readable code; strong AI-assisted tooling

Web framework	FastAPI	REST API with automatic OpenAPI documentation
Data access	SQLAlchemy 2.0	Typed models and queries; raw SQL where needed for reports
Migrations	Alembic, with one migration folder per module and plugin	Install and uninstall each unit's tables independently
Validation	Pydantic v2	Request, response and settings schemas
Background jobs	Celery with Redis	Retries, schedules, rate-limited external calls
Plugin loading	Python entry points and pluggy hooks	Discover modules and plugins; call extension points
Quality	pytest, Hypothesis, Ruff, mypy, import-linter	Tests, property tests, style, types, module boundaries
Web app	React with TypeScript	Browser interface
POS app	React with local SQLite, packaged for Android and Windows	Offline selling
Files	S3-compatible object storage	Receipts, attachments, exports
AI	LLM API behind an internal gateway	Model can be changed without changing modules

02 · EXTENSIBILITY

Module and plugin system

Ziira Core is built from three kinds of unit. The kernel is always present. Modules and plugins are packages that can be installed, enabled, disabled and uninstalled without changing other code.

Unit	What it is	Examples
Kernel	The platform every unit runs on. Cannot be removed.	Tenancy, authentication, users, roles and approvals, audit, events, workflow engine, numbering, settings, files, jobs, module and plugin registry, integration framework
Module	A complete business area with its own data, API, screens, permissions and events.	Products, Inventory, POS and Sales, CRM, Purchasing, Payments, Finance, People and Payroll, Command Center
Plugin	A smaller package that adds a capability to the kernel or a module through defined extension points.	A new payment provider, a bank statement format, a report, a receipt template, a connector, an industry-specific feature

Of the 12 modules in Ziira Core 1.0, three form the kernel (Users, Roles and Approvals; Audit and Activity; Integrations). The other nine are pluggable modules.

Manifest

Every module and plugin has a manifest that declares everything the kernel needs to know:

```
[unit]
name        = "inventory"
kind        = "module"                # module | plugin
version     = "1.0.0"
sdk         = ">=1.0,<2.0"            # kernel SDK versions supported
requires    = ["products>=1.0"]       # must be enabled first
optional    = ["purchasing", "pos"]   # extra features when present

[provides]
permissions = ["stock.view", "stock.adjust", "stock.transfer", "stock.count"]
publishes   = ["stock.moved", "stock.adjusted", "transfer.received"]
subscribes  = ["sale.posted", "goods.received", "sale.returned"]
api_prefix  = "/api/v1/inventory"
menus       = ["inventory"]
settings    = ["negative_stock_allowed", "count_variance_threshold"]
jobs        = ["expiry_check:daily", "balance_verify:nightly"]
migrations  = "migrations"
```

Module dependencies

Module	Requires	Optional (extra features)
Products and Pricing	Kernel	Inventory
Inventory and Branches	Products	Purchasing, POS
Payments and Reconciliation	Kernel	POS, CRM, Finance
POS and Sales	Products, Inventory, Payments	CRM (credit sales), Finance
CRM and Customers	Kernel	POS and Sales, Payments
Purchasing and Suppliers	Products	Inventory, Finance
Finance and Accounting	Kernel	All modules (posts their documents)
People, HR and Payroll	Kernel	Finance, Payments, POS (attendance links)
Command Center	Kernel	All modules (shows what is enabled)

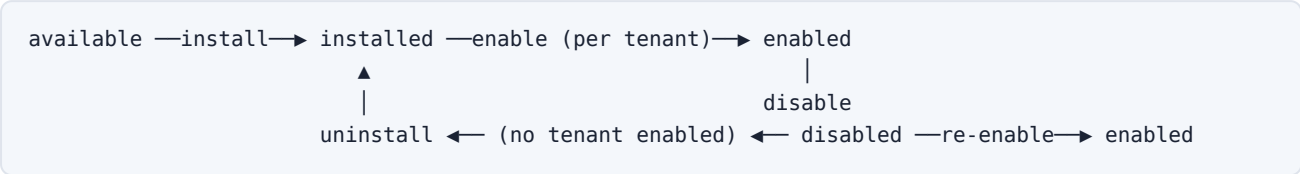
A module cannot be disabled while an enabled module requires it. Optional features switch on or off automatically when the other module is enabled or disabled.

How units talk to each other

- **Public interface:** each unit exposes a small `public.py` of functions and data types. Other units may call only these, never internal code or tables.
- **In-transaction events:** when a document is posted, the kernel dispatches its event to enabled subscribers inside the same database transaction. This is how Finance posts journals and Inventory moves stock for a sale, atomically. If Finance is not enabled, there is no subscriber and the sale still posts.

- **After-commit events:** events are also written to the outbox and processed after commit, for notifications, webhooks, plugins and Ziira Intelligence.

Lifecycle



Step	What the kernel does
Install	Verifies signature and SDK compatibility, checks dependencies, runs the unit's migrations, registers routes, hooks, permissions and jobs. Done once per platform.
Enable	Per tenant: checks required modules are enabled, seeds default data (accounts, reason codes, settings), adds permissions to default roles, starts jobs and subscriptions.
Disable	Per tenant: hides screens and menus, rejects its API calls, skips its event subscribers, pauses jobs. All data is kept.
Remove data	Per tenant, optional: export first, then delete the unit's data after a retention period. Not allowed for data the law requires to be kept, such as posted financial records.
Uninstall	Platform level, only when no tenant has it enabled: unregisters it and reverses its migrations.

Plugin extension points

Extension point	What a plugin can add
Payment provider	New ways to receive or send money (another mobile money service, card terminal)
Statement parser	A new bank or wallet statement format
Notification channel	A new channel such as a different SMS gateway
Import and export format	A new file format for data migration or accounting export
Report	A new report or dashboard widget
Document template	Receipt, invoice or payslip layouts
Workflow action	A new action that rules can call
Intelligence check	A new check for Ziira Intelligence
Custom fields	Extra fields on existing records (stored in the plugin's own schema)
UI slots	A menu item, a tab on a record, a button, a widget
Validation hook	A rule that can block an operation (for example, a sector-specific compliance check)

Rules for plugins

- Plugins use only the published SDK. They never import module internals or write to module tables directly.
- A plugin's tables live in its own PostgreSQL schema (`plg_<name>`) and are created and removed by its own migrations.
- A plugin requests permissions in its manifest; the tenant admin sees and approves them when enabling.
- Hooks run with time limits. A failing after-commit hook is logged and retried, and never breaks the core operation. Only validation hooks may block an operation.
- In 1.0, only plugins built and signed by Ziira can be installed. Third-party plugins can be opened later with a review process.

---

03 · CODE STRUCTURE

Code structure and standards

Every unit has the same layout and the same layers, so anyone who understands one module can follow all of them.

Repository layout

```
ziira/
  kernel/                 tenancy, auth, users, roles, audit, events, workflow,
                          numbering, settings, files, jobs, registry
  sdk/                    the only code modules and plugins may import from the kernel
  modules/
    inventory/
      manifest.toml
      public.py           functions and types other units may use
      api/                routes.py, schemas.py      (HTTP in and out only)
      services/           one file per use case, e.g. post_adjustment.py
      domain/             entities and business rules (no database, no HTTP)
      repository/         database access for this module only
      events.py           events published and handlers for events subscribed
      permissions.py
      migrations/
      tests/
      README.md
    products/  pos/  payments/  finance/  ...
  plugins/
    statement_parser_bankx/   same layout as a module, smaller
  web/                    React web app
  pos/                    React POS app
  docs/decisions/         short records of important design decisions
```

Layers inside a unit

Layer	Responsibility	May call
api	Receive requests, validate input, return responses	services
services	One use case per function: load, check rules, save, publish events	domain, repository, other units' public.py, sdk
domain	Business rules and calculations; pure Python, easy to test	nothing outside domain
repository	Read and write this unit's tables	database

Coding standards

- Type hints on every function; mypy in strict mode. Ruff for style and formatting.
- Module boundaries are enforced automatically: import-linter fails the build if a unit imports another unit's internals.
- Clear names over short names: `post_stock_adjustment()`, not `proc_adj()`.
- Small units: one use case per service file; functions that do one thing; no file above a few hundred lines without reason.
- Every public function has a docstring stating what it does, its inputs and its errors.
- Business rules live in `domain`, never in routes or SQL.
- Money uses `Decimal`, never float. Rounding is done by one shared helper.
- Every unit has a README: purpose, dependencies, events, permissions, settings.
- Every change comes with tests; a unit's tests run with only its required dependencies enabled.
- Important design choices are recorded in `docs/decisions` in a few paragraphs each.

04 · TENANCY

Tenancy and organisation model

Every customer is a tenant. All tenants share the same application and database; every row belongs to exactly one tenant and is isolated by the database itself.

```
Tenant (the customer account, billing)
 └─ Company (legal entity: KRA PIN, VAT status, chart of accounts, fiscal year)
     └─ Branch (outlet: address, eTIMS device, M-Pesa numbers, cash accounts)
         └─ Location (store, warehouse, shelf area, in-transit)
 Users belong to the tenant and are granted roles per company and branch.
```

Isolation

- Every tenant-owned table has a `tenant_id` column. PostgreSQL row-level security policies restrict every query to the tenant set on the connection, so a missing filter in code cannot leak data.

- The tenant is resolved from the authenticated session or API key, never from a request parameter.
- Background jobs carry the tenant context explicitly and set it before touching data.
- Large tenants can later be moved to a dedicated database without code changes, because tenant context is already explicit.

Company-level settings

Setting	Purpose
Base currency	KES by default; other currencies stored with exchange rate at transaction date
Fiscal year and periods	Monthly periods; open, soft-closed or locked
Tax registration	KRA PIN, VAT registered or not, eTIMS integration mode
Chart of accounts	Created from a Kenyan SME template, editable
Costing method	Weighted average cost (1.0 default)
Document series	Numbering per document type and branch
Policies	Negative stock allowed or not, credit limit enforcement, discount limits, approval thresholds

05 · DATA CONVENTIONS

Data conventions

These rules apply to every table and document. They make the data reliable for accounting, auditable, and ready for Ziira Intelligence.

Rule	Implementation
Database schemas	Each unit owns one PostgreSQL schema, e.g. `inventory`. All tables and columns are listed in Appendix A.
Identifiers	UUID v7 primary keys (time-ordered). Human document numbers are separate, e.g. `INV-THK-2026-000123`.
Standard columns	`tenant_id`, `company_id`, `branch_id`, `created_at`, `created_by`, `updated_at`, `updated_by`, `source` (web, pos, api, import, whatsapp, system), `status`.
Time	Stored in UTC; displayed in Africa/Nairobi. Business date stored separately from timestamp for documents.
Money	`numeric(18,2)` for amounts, `numeric(18,6)` for unit costs and rates. No floating point. Rounding rules defined once.
Status machines	Each document type has an explicit set of states and allowed transitions, enforced in one place.
Posted documents are immutable	Once a document is posted (affects stock or books), it is never edited. Corrections are made by a reversing or adjusting document linked to the original.

No hard deletes	Drafts may be discarded; anything posted is cancelled or voided, keeping the record and the reason.
Reason codes	Voids, refunds, discounts above policy, price overrides, stock adjustments and write-offs require a reason code from a managed list, plus optional note.
Creator and approver	Stored separately (`created_by`, `approved_by`, `approved_at`). The same user cannot be both where policy forbids it.
External references	Payments store method and external reference (M-Pesa receipt number, bank reference, cheque number), unique per tenant and method.
Links	Documents reference their origin: invoice to sales order, goods receipt to purchase order, payment to invoice, stock movement to source document.
Idempotency	Every create request from POS, API or integration carries an idempotency key, so retries never create duplicates.
History	Changes to master data (prices, customer credit limits, supplier bank details, employee pay) are kept as dated versions.

Document lifecycle

```
draft ──submit──▶ pending_approval ──approve──▶ approved ──post──▶ posted
  │                     │ reject                                    │
  └──discard            ▼                                           ├─▶ reversed (by a linked
reversal)
                    rejected                                        └─▶ voided (same-day POS only,
with reason)
```

Not every document needs every state; for example, a POS sale goes directly from in-progress to posted. Approval steps are inserted by the workflow engine according to the approval matrix (section 13).

06 · MODULES

Modules

For each module: its main entities, key workflows, and the events it publishes. Events feed other modules, the audit trail and Ziira Intelligence.

01 Command Center

Tables	`command.brief_schedule`, `command.dashboard_widget`, `intel.check_definition`, `intel.finding`, `intel.finding_event`, `intel.baseline` (`intel` belongs to Ziira Intelligence). Approvals use `kernel.approval_request`.
Workflows	Assemble pulse metrics per branch from read models; list findings and pending approvals for the current user's scope; schedule and send briefs; Ask Ziira queries.
Notes	Reads only from read models and the findings store; never computes from raw tables at page load.

02 Point of Sale and Sales

Tables	`pos.pos_device`, `pos.till_session`, `pos.sale`, `pos.sale_line`, `pos.sale_payment`, `pos.sales_document`, `pos.sales_document_line`, `pos.sales_invoice`, `pos.sales_invoice_line`, `pos.sales_return`, `pos.sales_return_line`
Workflows	Open till with float → sell (scan, price, discount within limit, take payment) → receipt with eTIMS details → close till with cash count. B2B: quotation → sales order → delivery → invoice → payment. Returns create a credit note and restock.
Events	`sale.posted`, `sale.voided`, `sale.returned`, `discount.override`, `till.opened`, `till.closed`, `invoice.issued`

03 CRM and Customers

Tables	`crm.customer`, `crm.customer_contact`, `crm.customer_credit_version`, `crm.lead_source`, `crm.lead`, `crm.activity`
Workflows	Capture lead → follow-ups → convert to customer. Set credit limit and terms (versioned, approval above threshold). Statements and ageing. Reminders by WhatsApp, SMS or email.
Events	`lead.created`, `lead.converted`, `customer.credit_limit_changed`, `customer.over_limit_attempt`

04 Products and Pricing

Tables	`products.category`, `products.uom`, `products.tax_code`, `products.product`, `products.product_uom`, `products.barcode`, `products.price_list`, `products.price_list_item`
Workflows	Create product with units and tax code; price lists per branch or customer group with effective dates; bulk price updates with approval.
Events	`price.changed`, `product.created`

05 Inventory and Branches

Tables	`inventory.location`, `inventory.batch`, `inventory.stock_movement`, `inventory.stock_balance`, `inventory.product_cost`, `inventory.transfer`, `inventory.transfer_line`, `inventory.stock_adjustment`, `inventory.stock_adjustment_line`, `inventory.stock_count`, `inventory.stock_count_line`
Workflows	Every quantity change is a movement. Transfers: dispatch (to in-transit) → receive (with variance). Counts: freeze snapshot → count → variance review → post adjustment with approval.
Events	`stock.moved`, `stock.adjusted`, `transfer.dispatched`, `transfer.received`, `count.posted`, `stock.below_reorder`

Engine details in section 08.

06 Purchasing and Suppliers

Tables	`purchasing.supplier`, `purchasing.supplier_bank_account`, `purchasing.purchase_request`, `purchasing.purchase_request_line`, `purchasing.purchase_order`, `purchasing.purchase_order_line`, `purchasing.goods_receipt`, `purchasing.goods_receipt_line`, `purchasing.supplier_bill`, `purchasing.supplier_bill_line`, `purchasing.supplier_price`
Workflows	Request → approved purchase order → goods receipt (partial allowed) → supplier bill matched to order and receipt (three-way match, tolerance configurable) → payment.
Events	`po.approved`, `goods.received`, `bill.posted`, `bill.match_exception`, `supplier.bank_changed`, `supplier.price_changed`

07 Payments and Reconciliation

Tables	`payments.payment_account`, `payments.external_transaction`, `payments.payment_intent`, `payments.payment`, `payments.payment_allocation`, `payments.match`, `payments.daily_close`, `payments.recon_exception`
Workflows	Receive payment notifications and statements → normalise into external transactions → match to sales and invoices → flag exceptions → branch daily close.
Events	`payment.received`, `payment.matched`, `payment.unmatched`, `close.completed`, `close.variance`

Engine details in section 10.

08 Finance and Accounting

Tables	`finance.account`, `finance.period`, `finance.posting_rule`, `finance.journal`, `finance.journal_entry`, `finance.journal_line`, `finance.expense`, `finance.bank_reconciliation`. Receivables and payables are derived from invoices, bills and payment allocations.
Workflows	Automatic posting from operational documents; manual journals with approval; expense claims with receipt and approval; period close; VAT summary; export to accounting software.
Events	`journal.posted`, `journal.manual_posted`, `period.closed`, `expense.approved`

Engine details in section 07.

09 People, HR and Payroll

Tables	`people.department`, `people.employee`, `people.pay_component`, `people.employment_terms`, `people.employee_pay_item`, `people.employee_bank_account`, `people.shift`, `people.attendance`, `people.leave_type`, `people.leave_request`, `people.statutory_rate_table`, `people.payroll_run`, `people.payslip`, `people.payslip_line`

Workflows	Onboard employee; record attendance (linked to POS user and till sessions); leave requests and approval; monthly payroll run → review → approve → post → pay → statutory files.
Events	`employee.pay_changed`, `employee.bank_changed`, `payroll.approved`, `payroll.paid`

Engine details in section 12.

10 Users, Roles and Approvals · 11 Audit and Activity · 12 Integrations

These are platform modules, described in sections 13, 14 and 15.

07 · ACCOUNTING ENGINE

Accounting engine

The ledger is double-entry. Operational modules never write journal lines directly; they publish posted documents, and the posting engine converts them into balanced journal entries using configurable posting rules.

Principles

- Every journal entry balances: total debits equal total credits, enforced by a database constraint at commit.
- Every journal line carries `company_id`, `branch_id`, account, amount and a link to its source document, so profit and loss can be reported per branch.
- Posting happens in the same database transaction as the source document, so stock, payments and books never disagree.
- Posted entries are immutable. Corrections are reversing entries.
- Sub-ledgers (customers, suppliers, inventory, M-Pesa clearing) must equal their control accounts. A nightly job checks this and raises an exception if not.

Core posting rules

Event	Debit	Credit
Cash sale	Till cash (branch)	Sales revenue; VAT output
M-Pesa sale	M-Pesa clearing (till or Paybill)	Sales revenue; VAT output
Credit sale or invoice	Accounts receivable	Sales revenue; VAT output
Cost of any sale	Cost of goods sold	Inventory
Customer payment	M-Pesa clearing or bank	Accounts receivable
Sales return	Sales returns; VAT output; Inventory	Cash, clearing or receivable; COGS

Goods received	Inventory	Goods received not invoiced
Supplier bill	Goods received not invoiced; VAT input	Accounts payable
Supplier payment	Accounts payable	Bank or M-Pesa
Expense	Expense account; VAT input	Cash, bank or payable
Stock loss or write-off	Inventory shrinkage expense	Inventory
Branch transfer	Inventory in transit, then destination inventory	Source inventory, then in transit
Cash deposit to bank	Bank	Till cash
M-Pesa settlement	Bank; M-Pesa charges	M-Pesa clearing
Till shortage	Cash shortage expense or staff receivable	Till cash
Payroll	Salaries; employer statutory costs	Net pay payable; PAYE, SHIF, NSSF and Housing Levy payables; other deductions
Salary payment	Net pay payable	Bank or M-Pesa

Posting rules are data, not code: each maps a document type and line type to accounts, with defaults from the chart of accounts template and per-company overrides.

Periods and closing

- Periods are monthly. A *soft-closed* period accepts entries only from users with close permission; a *locked* period accepts none. Back-dated documents into a closed period are rejected or redirected to the current period, by policy.
- Period close checklist: all branch days closed, reconciliations complete, sub-ledgers equal control accounts, payroll posted, depreciation and accruals entered manually where needed.

Reports

Trial balance, profit and loss (by branch and consolidated), balance sheet, cash flow summary, general ledger detail, customer and supplier ageing, VAT summary. Reports read from journal lines and are reproducible for any past date.

Export to accounting software

Journal export by period in CSV and in formats importable by QuickBooks and Sage, with account mapping per company, for businesses whose accountant keeps a separate system.

Inventory engine

Stock is a ledger. Quantities are never edited directly; each change is an immutable movement, and balances are derived from movements.

Stock movement record

```
inventory.stock_movement   (full definition in Appendix A)
  id, tenant_id, created_at, created_by
  company_id, branch_id, location_id, product_id, batch_id
  quantity        + in / − out, in base unit
  unit_cost, value
  movement_type   sale | sale_return | receipt | purchase_return | transfer_out
                  | transfer_in | adjustment | count_variance | write_off | opening
  source_type, source_id, source_line_id
  reason_code_id  required for adjustment, count_variance, write_off
  business_date, approved_by
```

- `stock_balance` per location, product and batch is maintained in the same transaction as each movement and can be rebuilt from movements at any time. A nightly job verifies they match.
- Concurrency: balance rows are locked per product and location during posting to prevent double-selling the last unit.

Costing

- Weighted average cost per product per company in 1.0. On each receipt: new average = (existing value + receipt value) ÷ (existing quantity + receipt quantity).
- Sales and outbound movements take the current average cost; the value posts to cost of goods sold.
- Landed costs (transport, duty) can be added to a receipt and spread across its lines before averaging.

Batches and expiry

- Products can be batch-tracked. Batch-tracked items pick first-expiry-first-out by default; the cashier can override with a reason.
- Expiry warnings are generated from batch quantities and dates.

Negative stock

A company policy. If allowed, the sale posts and an exception is raised for review; the cost is corrected when stock arrives. If not allowed, the sale is blocked. Offline POS sales are always accepted and reviewed after sync (section 09).

Transfers and counts

- Transfer dispatch moves stock from source to an in-transit location; receipt moves it to the destination. Differences on receipt require a reason and approval.
- Stock counts take a snapshot at start. Movements during the count are tracked, so the variance is calculated against the right expected figure. Variances above a threshold need approval before posting.

09 · POINT OF SALE

Point of sale and offline operation

The POS must keep selling when the internet fails. It keeps a local copy of what it needs, records sales locally, and synchronises when connected.

Device and session

- Each POS device is registered to a branch and authenticated with a device credential plus the cashier's login (PIN or password).
- A till session is opened with a cash float and closed with a counted cash amount. Expected cash = float + cash sales − cash refunds − cash paid out. The difference is recorded and, above tolerance, raises an exception and a posting to till shortage or overage.
- X report (mid-session summary) and Z report (session close) are produced per till session.

Offline design

Concern	Approach
Local data	Products, prices, tax codes, customers with credit status, cashier permissions and current stock snapshot for the branch, in local SQLite. Refreshed incrementally.
Recording sales	Sales are written locally with a device-generated UUID and a local sequence number, then queued for upload.
Numbering	Each device has its own document series (e.g. `POS-THK-02-000451`), so offline devices never collide.
Sync	Uploads use idempotency keys; the server posts each sale exactly once, in device order. Downloads apply changes since the last sync.
Conflicts	Sales are facts and are always accepted. If a price, stock level or credit limit changed while offline, the sale posts as recorded and an exception is raised for review.
M-Pesa offline	STK push needs connectivity. Offline, the cashier records the M-Pesa receipt code shown on the customer's message; matching confirms it later.
eTIMS offline	Sales are queued for eTIMS submission and submitted on reconnection, within the period the eTIMS rules allow (section 11).

Limits	A configurable maximum offline duration and sale count, after which the device warns and then stops selling until it syncs.

Hardware

Receipt printers (ESC/POS), barcode scanners (keyboard input), cash drawers (via printer), customer displays optional. Android tablets and Windows PCs are the primary targets.

10 · PAYMENTS AND RECONCILIATION

Payments and reconciliation

All money received or paid is normalised into one model, then matched to the documents it belongs to. Unmatched items become exceptions for people and for Ziira Intelligence.

M-Pesa integration (Safaricom Daraja)

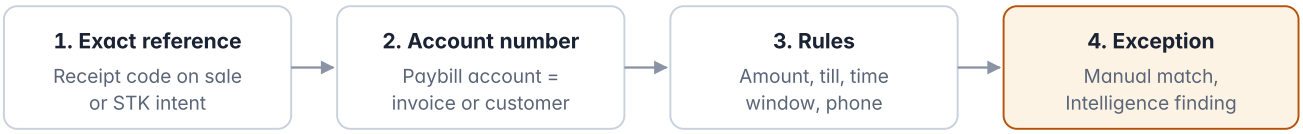
Capability	Use in Ziira Core
STK push (Lipa na M-Pesa Online)	Cashier sends a payment prompt to the customer's phone; the result callback completes the payment intent.
C2B confirmation and validation	Notifications for payments customers make directly to the Till or Paybill. Requires registering Ziira's URLs on the shortcode; only one set of URLs can be registered per shortcode.
Transaction status query	Confirms a payment when a callback was missed or delayed.
Pull transactions	Recovers missed C2B transactions; limited to a recent window (48 hours).
B2C	Salary or supplier payouts to phones, where the business has a B2C shortcode.
Statements	Uploaded M-Pesa business statements fill gaps and provide history.

Each business connects its own shortcodes with its own Daraja credentials, stored encrypted. If a shortcode's C2B URLs are already used by another system, Ziira relies on STK push, status queries and statements for that shortcode.

Banks

Bank APIs where the bank provides them; otherwise statement import (CSV and common bank formats, with PDF extraction as a fallback). Each imported line becomes an `external_transaction`.

Matching engine



Each stage runs only on items not matched by the previous one. Rule matches below a confidence threshold are proposed, not applied. Every match records the stage, rule and user.

- One external transaction can match several documents (one payment for three invoices), and one document can be paid by several transactions (part payments).
- Matches can be undone by authorised users; the history is kept.

Daily branch close

1. All till sessions for the branch and business date are closed.
2. M-Pesa and card totals are matched against sales paid by those methods.
3. Cash banked is recorded against expected cash.
4. Variances are listed with their causes; the branch manager submits the close, with explanations where required.
5. The close is locked. Later changes for that date require a reopening with approval.

---

11 · ETIMS

eTIMS integration

Kenyan businesses must issue tax invoices through KRA's electronic Tax Invoice Management System (eTIMS). Ziira Core issues eTIMS-compliant invoices for every sale and credit note.

Integration approach

- Ziira integrates directly with KRA's eTIMS through KRA's system-to-system integration interface.
- Before going live, Ziira completes KRA's onboarding for system integrators, including testing and certification in KRA's test environment, and maintains that status.
- Each company and branch stores its KRA PIN and eTIMS registration details, captured during onboarding.
- The eTIMS connector is a kernel integration used by POS and Sales and Finance; it is not optional for VAT-registered businesses.

Flow

```
sale or invoice posted
  → eTIMS submission job (queue, retried with backoff)
  → KRA response: control unit signature, invoice number, QR data
  → stored on the document and printed on the receipt
failures → document flagged "eTIMS pending"; visible in Command Center;
           resubmitted automatically; alert if not cleared within the allowed time
```

Requirements on the data

- Every product carries its eTIMS item classification and tax type.
- Credit notes reference the original invoice's eTIMS number.
- Purchases from eTIMS-registered suppliers can be pulled from eTIMS and matched to supplier bills where the integration allows.
- Item, customer and invoice data sent to KRA follow the current eTIMS technical specification, which the team must obtain from KRA and track for changes.

---

12 · PAYROLL

Payroll engine

Payroll runs monthly per company. Statutory rules are held in effective-dated rate tables, not in code, because Kenyan rates and rules change.

Inputs

- Employment terms (basic pay, allowances, benefits), versioned with effective dates.
- Attendance, overtime and unpaid leave for the period.
- Recurring and one-off earnings and deductions (advances, loans, SACCO, union dues).
- Statutory rate tables: PAYE bands and reliefs, SHIF, NSSF tiers and limits, Affordable Housing Levy, and any other statutory items in force.

Calculation pipeline

```
1 gross pay      = basic + allowances + overtime + taxable benefits − unpaid leave
2 statutory pre-tax deductions (per rate tables, in the order the law sets)
3 taxable pay    = gross − allowable deductions
4 PAYE           = band calculation − reliefs
5 other statutory and voluntary deductions
6 net pay        = gross − all deductions
7 employer costs = employer NSSF, employer Housing Levy, other employer items
```

Which deductions reduce taxable pay, and in what order, is part of the rate table definition, so a change in law is a data change with an effective date.

Run lifecycle

1. Create run for the period → calculate → review exceptions (new employees, pay changes, bank changes, net pay below threshold).
2. Approve (approver cannot be the person who prepared the run, by policy).
3. Post to the ledger (section 07).
4. Pay: bank bulk payment file, or M-Pesa B2C where configured.
5. Outputs: payslips (PDF, sent by WhatsApp or email), P9 forms, and statutory return files for upload to the relevant portals (KRA iTax, SHA, NSSF, Housing Levy).

> **Correctness:** current rates must be confirmed from official sources at implementation and whenever they change. Before launch, results are checked against a set of test employees calculated independently by an accountant, covering every band and edge case.

13 · ACCESS CONTROL

Access control and workflow

Permissions

- Permissions are fine-grained actions, e.g. `sales.void`, `stock.adjust`, `price.override`, `journal.post_manual`, `payroll.approve`, `supplier.bank_edit`.
- Roles are named sets of permissions (Owner, Finance Manager, Branch Manager, Cashier, Storekeeper, Sales Rep, HR, Accountant read-only). Tenants can create custom roles.
- A role is granted to a user with a **scope**: whole company, specific branches, or own records only. Every query and action checks both permission and scope.
- Sensitive data (payroll, employee bank details) requires explicit permissions not included in general manager roles.

Approval matrix

Document or action	Condition	Approver
Purchase order	Above amount threshold	Role, or next level above a second threshold
Discount or price override	Above the cashier's limit	Branch manager (PIN on POS)
Stock adjustment	Any, or above value threshold	Branch manager or owner
Credit limit change	Any increase	Finance manager
Supplier bank change	Always	Finance manager, plus confirmation step
Manual journal	Always	Finance manager
Payroll run	Always	Owner or director

The matrix is configurable per company. **Separation of duties** is enforced: where configured, the creator of a document cannot approve it. Approvals can be given in the web app, on POS (manager PIN), or on WhatsApp (section 17).

Workflow rules

A rule engine runs cross-module automation defined as *trigger → conditions → actions*:

```
trigger:    invoice.overdue (days_overdue = 7)
conditions: customer.segment = "wholesale" AND balance > 50000
actions:    send_reminder(channel = whatsapp, template = "overdue_7")
            create_task(assignee = account_manager)
```

Triggers are domain events or schedules; actions are a fixed, permission-checked set (notify, create task, request approval, send reminder, set status). Rules are versioned and every execution is logged.

14 · AUDIT

Audit trail

Every significant action is recorded in an append-only audit log that cannot be edited through the application.

Recorded	Detail
Record changes	Who, when, from which device and channel, previous and new values of changed fields
Document transitions	Submitted, approved, rejected, posted, reversed, voided, with reason codes
Approvals and overrides	Who approved what, at which threshold, through which channel
Security events	Logins, failed logins, MFA, role changes, permission grants, API key creation, data exports
Integration activity	Inbound and outbound calls with status, linked to affected records
Intelligence actions	Every message sent and action taken by Ziira, and who authorised it

Integrity

- The application database user has insert-only rights on audit tables.
- Each entry includes a hash of the previous entry for the tenant, so tampering breaks the chain and is detectable.
- Retention: at least seven years for financial records, configurable longer.

API, events and integrations

REST API

- Versioned JSON API (`/api/v1/...`); the web app and POS use the same API as third parties.
- Authentication: user sessions for the web app; OAuth 2.0 for third-party apps acting for a user; scoped API keys for server-to-server integrations.
- Conventions: cursor pagination, filtering by updated time for incremental sync, idempotency keys on create, consistent error format, rate limits per key.
- An OpenAPI description is generated from the code and published.

Events and webhooks

- Domain events are written to an outbox table in the same transaction as the change, then published to the internal bus. This guarantees no event is lost or published for a change that rolled back.
- Tenants can subscribe external URLs to events. Webhooks are signed (HMAC), retried with backoff, and logged.

Imports and exports

- Guided imports with validation preview for products, customers, suppliers, opening stock, opening balances and employees. Used for onboarding and migration from other systems.
- Exports of any list and report to CSV and Excel; accounting journal export (section 07).

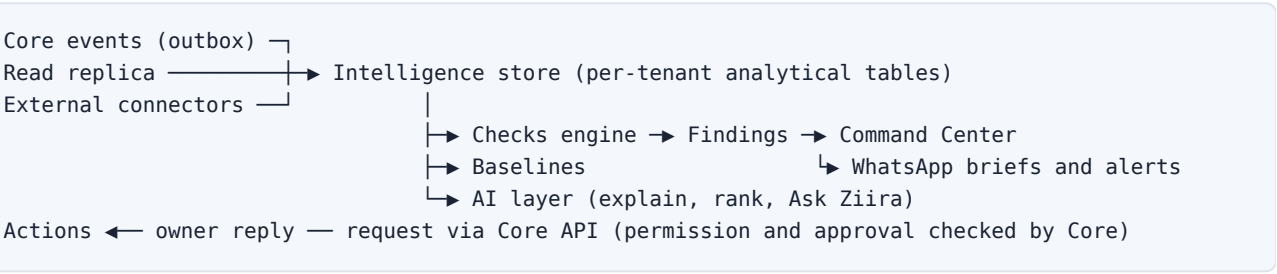
Connector list for 1.0

Connector	Direction and purpose
Safaricom Daraja	Payments in and out, status, statements (section 10)
KRA eTIMS	Tax invoices and credit notes (section 11)
Banks	Statements in; payment files out
Meta WhatsApp Cloud API	Briefs, alerts, reminders, approvals, payslips (section 17)
Email and SMS gateway	Notifications, statements, one-time passwords
Accounting export	Journals to QuickBooks, Sage and CSV

Ziira Intelligence interface

Ziira Intelligence is a separate service that reads Core data, produces findings, and requests actions through the Core API. The same service also runs standalone on other systems through connectors that map their data to the same model.

Data flow



Components

Component	Function
Canonical model	A common schema for sales, payments, stock movements, documents, users and branches. Core maps to it directly; connectors map Odoo, Zoho, QuickBooks and others to it.
Checks engine	Deterministic checks written as versioned SQL or code: reconciliation gaps, control checks, thresholds. Each check has an ID, severity, scope and evidence query.
Baselines	Per-tenant normal ranges (e.g. void rate per cashier, daily sales per branch and weekday) computed from history, used to flag unusual values.
Findings store	Each finding: check ID, entity, period, value, expected value, evidence record IDs, severity, status (new, sent, acknowledged, explained, resolved, dismissed), owner.
AI layer	Writes the plain-language explanation from the finding and its evidence; ranks findings; answers questions by calling defined query tools, never by writing free SQL against production.
Action requests	Proposed actions (send reminder, draft PO, ask staff) are submitted to the Core API as the requesting user, so Core's permissions and approval matrix apply.

Rules for the AI layer

- Figures shown to users come from checks and queries, never generated by the model.
- Ask Ziira answers are limited to the asking user's permissions and branch scope.
- Every AI response stores the tools called and data used, for review.
- Tenant data is not used to train external models; the AI provider's data terms must guarantee this.

17 · WHATSAPP

WhatsApp channel

WhatsApp is a channel into Core and Intelligence, not a separate system. Messages read from and act through the same API and permissions as the web app.

Setup

- Uses the Meta WhatsApp Business Cloud API. Ziira operates a verified business number; tenants may later connect their own number.
- Business-initiated messages outside the 24-hour customer service window must use pre-approved message templates (briefs, alerts, reminders, payslip notices). Replies within the window can be free-form.
- Meta charges per message by category; costs are tracked per tenant.

Identity

- A phone number is linked to a Ziira user only after verification (one-time code from the web app). Unknown numbers receive no business data.
- Each message is processed with that user's permissions and scope.

Approvals by WhatsApp

- Approval requests include the document summary and a one-time reference. The user replies with the option; high-value approvals also require a PIN.
- Approvals expire after a configurable time. Every WhatsApp approval is logged in the audit trail with the message ID.

Staff reports

Staff can send closing figures, stock counts and photos of receipts. Ziira parses them into draft records, which are confirmed by the sender and reviewed by the branch manager before posting.

18 · SECURITY

Security and data protection

Area	Measures
Authentication	Email or phone with password; one-time codes; MFA required for owners, finance and admin roles; device credentials for POS.
Sessions	Short-lived access tokens, refresh tokens bound to device, remote sign-out, idle timeout.

Encryption	TLS for all traffic. Encryption at rest for database, backups and files. Secrets (Daraja, bank, eTIMS credentials) in a secrets manager or encrypted with per-tenant keys.
Isolation	Row-level security by tenant (section 04); separate credentials per environment.
Application security	Input validation, parameterised queries, dependency scanning, security headers, rate limiting, protection against common web attacks.
Access by staff	Ziira staff access to tenant data only with tenant consent, time-limited and logged.
Testing	External penetration test before launch and yearly.

Kenya Data Protection Act

- Register with the Office of the Data Protection Commissioner as required for Ziira's role (data processor for tenants' data; controller for its own customer data).
- Data processing agreement with every tenant.
- Data minimisation, purpose limitation and retention rules; support for data subject requests (access, correction, deletion where lawful).
- Assess and document any cross-border transfers, including to AI providers.
- Breach response procedure, including notification within the time the law requires.

19 · INFRASTRUCTURE

Infrastructure and operations

Area	Approach
Environments	Development, staging (with Daraja and eTIMS sandboxes) and production, fully separated.
Deployment	Docker containers; automated pipeline: tests, migrations, deploy; zero-downtime releases; migrations backward-compatible for one release.
Backups	Continuous write-ahead log archiving for point-in-time recovery; daily snapshots kept 35 days; monthly kept 12 months; restore tested monthly.
Recovery targets	Target data loss no more than 15 minutes; target restoration within 4 hours.
Monitoring	Uptime checks, error tracking, performance metrics, queue depth, integration failure rates, alerting to on-call.
Logging	Structured logs with tenant and request IDs; no secrets or full card or ID numbers in logs.
Scaling	Stateless application servers scale horizontally; reports and Intelligence read from the replica; heavy jobs run in workers.

20 · TESTING

Testing and quality

Accounting, stock and payroll errors are expensive and damage trust. Correctness is proven with tests that run on every change.

Test type	What it proves
Posting rule tests	Every document type produces the expected balanced journal entry
Invariant tests	Random sequences of transactions always keep: debits = credits, sub-ledgers = control accounts, stock balance = sum of movements
Payroll golden cases	Results match independently calculated cases for every band and edge case
Costing tests	Weighted average cost is correct across receipts, returns, transfers and negative stock
Offline POS tests	Sales made offline on several devices sync once, in order, with no duplicates or gaps
Integration tests	Daraja and eTIMS flows in sandboxes, including timeouts, duplicates and missed callbacks
Permission tests	Each role can do exactly what it should; no cross-tenant or cross-branch access
Migration tests	Imports of real sample data from other systems reconcile to source totals
Module lifecycle tests	Each module and plugin installs, enables, disables and uninstalls cleanly; disabling one never breaks another
Architecture tests	Import rules between units are respected (import-linter)
End-to-end tests	Main journeys: open till, sell, close day, receive stock, pay supplier, run payroll, close period

AI-generated code is held to the same standard: it is merged only with passing tests, and posting rules, payroll and permissions code is reviewed by a person who understands the domain.

21 · BUILD PLAN

Build plan

Modules are built in dependency order. Each phase ends with exit criteria that must pass before the next starts.

#	Phase	Scope	Exit criteria
1	Foundation	Kernel: tenancy, auth, users, roles, scopes, audit, numbering, events and outbox, module and plugin registry, SDK, CI/CD	Isolation, permission and module lifecycle tests pass

2	Catalogue and inventory	Products, units, prices, tax codes, locations, stock movements, transfers, counts, costing	Stock and costing invariants pass
3	Ledger	Chart of accounts, posting engine, journals, periods, trial balance	Posting and balance invariants pass
4	Point of sale and payments	POS with offline, till sessions, STK push, C2B, matching, daily close	A branch can trade a full day offline and online, and close with all money matched
5	eTIMS	Invoice submission, credit notes, retry	Sandbox certification tests pass
6	Customers and sales	CRM, credit, invoices, receivables, statements, reminders	Credit sale to payment flow reconciles
7	Purchasing	Suppliers, orders, receipts, bills, three-way match, payables	Purchase to payment flow reconciles
8	Finance reports	Expenses, bank reconciliation, P&L, balance sheet, VAT summary, export	Reports reconcile to journal lines
9	People and payroll	Employees, attendance, leave, payroll, payslips, statutory files	All golden payroll cases pass
10	Command Center, Intelligence, WhatsApp	Read models, checks, findings, briefs, Ask Ziira, approvals	Pilot businesses receive correct daily briefs
11	Hardening	Performance, security test, backups, monitoring, migration tools	Penetration test issues fixed; restore test passes

Ziira Intelligence on external systems can be built in parallel from phase 3 onward, since it shares the canonical model and checks engine.

22 · OPEN DECISIONS

Open decisions

These need an owner and a decision before or early in the build.

#	Decision	Options and considerations
1	AI provider	Quality, cost per tenant, data terms (no training on tenant data), data location
2	WhatsApp number model	One Ziira number for all tenants, or tenant-owned numbers; affects branding and setup effort
3	POS packaging	Android and Windows apps, or installable web app; affects printer and cash drawer support
4	Payroll compliance review	Who signs off statutory calculations and keeps rate tables current
5	Plugin distribution	When to allow third-party plugins, and the review process for them

6	Pricing model	Per branch, per user, per company or per module; determines how usage is metered

> **Next step:** start phase 1 (kernel and module framework). Decisions 1 to 6 can be made during phases 1 to 4.

APPENDIX A · DATABASE

Database schema

The complete PostgreSQL schema for Ziira Core 1.0: 11 schemas and 110 tables. It is delivered as one migration per unit, generated from a single definition file so this appendix and the SQL always match.

Schema rules

- **One PostgreSQL schema per unit:** `kernel`, `products`, `crm`, `inventory`, `payments`, `pos`, `purchasing`, `finance`, `people`, `command`, and `intel` for the Ziira Intelligence store. Plugins get their own schema named `plg_<name>`.
- **Foreign keys only to the kernel and to required modules.** References to optional modules are stored as plain IDs (soft references), so an optional module can be uninstalled without touching any other schema.
- **Tenant isolation:** every tenant table has `tenant_id` with row-level security forced on. The application sets `app.tenant_id` per transaction; rows of other tenants are invisible and cannot be written.
- **Immutable records:** `inventory.stock_movement`, `finance.journal_entry`, `finance.journal_line` and `kernel.audit_log` reject updates and deletes at database level.
- **Balanced journals:** a deferred constraint trigger rejects any transaction that leaves a journal entry unbalanced.
- **Reason codes enforced:** stock adjustments, write-offs, count variances and voided sales cannot be saved without a reason code.
- **Allowed values** are check constraints on text columns, not database enum types, so units can extend them in their own migrations.

Standard columns

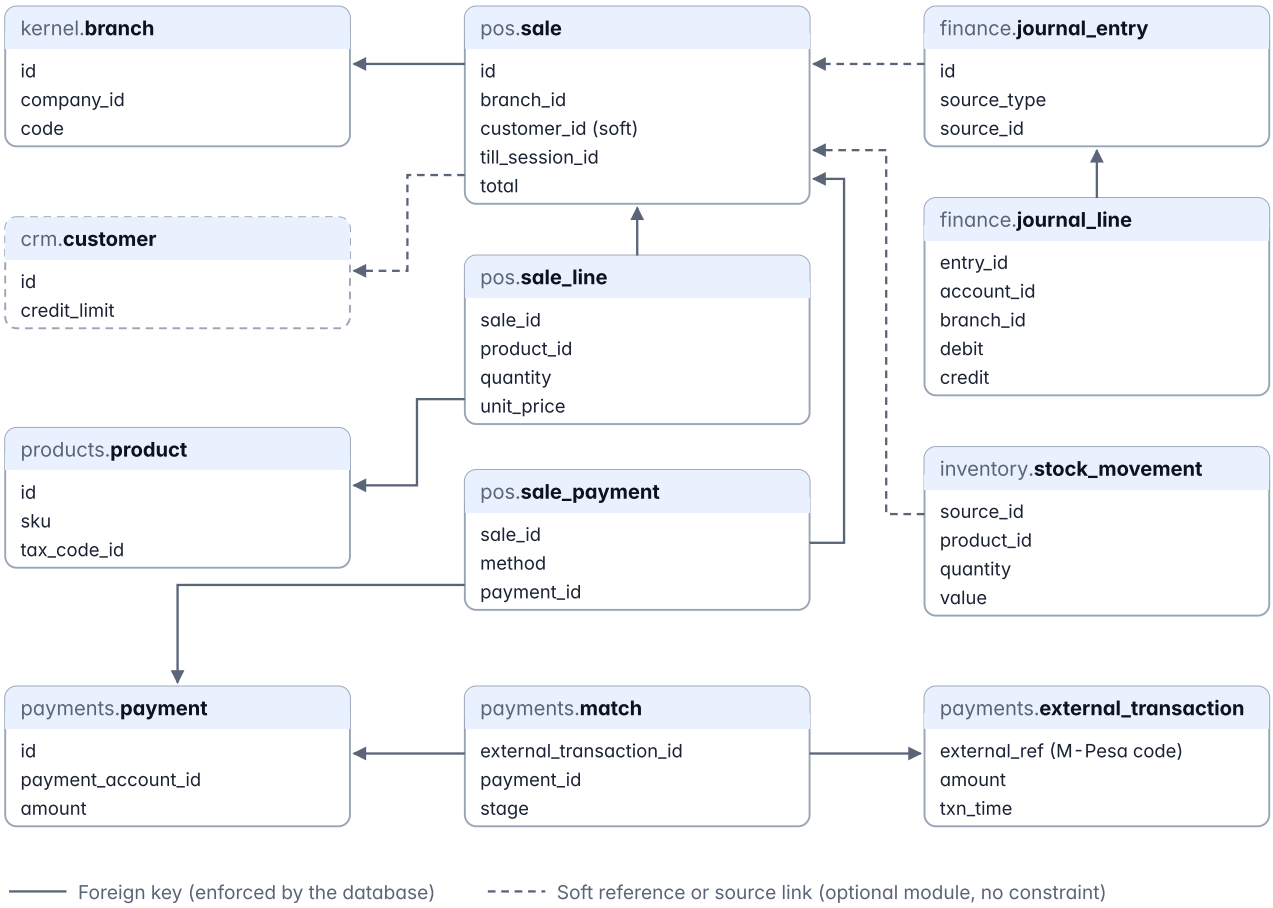
Not repeated in the table listings below.

Table kind	Columns added automatically
Tenant table (default)	`id` uuid primary key, `tenant_id` → kernel.tenant, `created_at`, `created_by` → kernel.app_user, `updated_at`, `updated_by` → kernel.app_user
Line table	`id`, `tenant_id`, `created_at` (lines belong to a parent document that carries the rest)

Platform table	None. Shared reference data managed by Ziira, without tenant_id
System table	Columns listed in full (outbox, audit log, idempotency keys)

Types: *money* = numeric(18,2); *quantity* = numeric(18,4); *rate* = numeric(18,6); *timestamp* = timestamptz (UTC).

Core relationships: a sale



Posting a sale creates the sale, its lines and payments, the stock movements and the journal entry in one transaction. Stock movements and journal entries point back to the sale through `source_type` and `source_id`. They also reference `products.product` and `kernel.branch` directly.

Testing

The migrations were installed on PostgreSQL 16 and tested for: all 110 tables created; tenant isolation (a second tenant sees and writes nothing of the first); rejection of an unbalanced journal; rejection of updates and deletes on immutable tables; rejection of a stock adjustment without a reason; and uninstalling and reinstalling CRM, Purchasing, People, Command Center and Finance without affecting any other schema.

Tables by schema

Schema `kernel` · Kernel

Platform tables. Always installed; cannot be removed. Requires: nothing. Migration: `000_kernel`.

`kernel.tenant` A customer account. root table

Column	Type	Req.	Notes
id	uuid	yes	primary key
name	text	yes	
status	text	yes	one of: active, suspended, closed; default 'active'
created_at	timestamp	yes	default now()

`kernel.app_user` A person who can sign in.

Column	Type	Req.	Notes
email	text		
phone	text		
full_name	text	yes	
password_hash	text		
mfa_enabled	boolean	yes	default false
status	text	yes	one of: invited, active, disabled; default 'active'
last_login_at	timestamp		

Constraints: unique (tenant_id, email); unique (tenant_id, phone)

`kernel.company` Legal entity with its own books.

Column	Type	Req.	Notes
name	text	yes	
kra_pin	text		
vat_registered	boolean	yes	default false
base_currency	char(3)	yes	default 'KES'
fiscal_year_start_month	smallint	yes	default 1
costing_method	text	yes	one of: weighted_average; default 'weighted_average'
settings	jsonb	yes	default '{}'; Policies: negative stock, discount limits, etc.

kernel.branch An outlet or site of a company.

Column	Type	Req.	Notes
company_id	uuid	yes	→ kernel.company
code	text	yes	Short code used in document numbers, e.g. THK
name	text	yes	
address	text		
phone	text		
is_active	boolean	yes	default true

Constraints: unique (company_id, code)

kernel.unit Installed modules and plugins (platform level). platform table, no tenant

Column	Type	Req.	Notes
name	text	yes	primary key
kind	text	yes	one of: module, plugin
version	text	yes	
manifest	jsonb	yes	
installed_at	timestamp	yes	default now()

kernel.tenant_unit Which units each tenant has enabled.

Column	Type	Req.	Notes
unit_name	text	yes	→ kernel.unit
status	text	yes	one of: enabled, disabled
settings	jsonb	yes	default '{}'
enabled_at	timestamp		
disabled_at	timestamp		

Constraints: unique (tenant_id, unit_name)

kernel.permission Permission codes declared by units (platform level). platform table, no tenant

Column	Type	Req.	Notes
code	text	yes	primary key; e.g. stock.adjust
unit_name	text	yes	→ kernel.unit
description	text	yes	

kernel.role Named set of permissions.

Column	Type	Req.	Notes
name	text	yes	
is_system	boolean	yes	default false; Default roles that cannot be deleted

Constraints: unique (tenant_id, name)

kernel.role_permission Permissions in a role. line table

Column	Type	Req.	Notes
role_id	uuid	yes	→ kernel.role
permission_code	text	yes	→ kernel.permission

Constraints: unique (role_id, permission_code)

`kernel.user_role` Role granted to a user with a scope.

Column	Type	Req.	Notes
user_id	uuid	yes	→ kernel.app_user
role_id	uuid	yes	→ kernel.role
company_id	uuid		→ kernel.company; Null = all companies
branch_id	uuid		→ kernel.branch; Null = all branches in scope
scope	text	yes	one of: tenant, company, branch, own; default 'branch'

Constraints: unique (user_id, role_id, company_id, branch_id)

`kernel.user_phone` Verified phone numbers for WhatsApp.

Column	Type	Req.	Notes
user_id	uuid	yes	→ kernel.app_user
phone	text	yes	
verified_at	timestamp		

Constraints: unique (tenant_id, phone)

`kernel.api_key` Keys for server-to-server integrations.

Column	Type	Req.	Notes
name	text	yes	
key_hash	text	yes	unique
scopes	text[]	yes	default '{}'
expires_at	timestamp		
revoked_at	timestamp		

`kernel.document_series` Numbering per document type and branch.

Column	Type	Req.	Notes
company_id	uuid	yes	→ kernel.company
branch_id	uuid		→ kernel.branch
doc_type	text	yes	e.g. sale, invoice, po
prefix	text	yes	e.g. INV-THK-
next_number	bigint	yes	default 1
padding	smallint	yes	default 6

Constraints: unique (company_id, branch_id, doc_type)

`kernel.reason_code` Managed list of reasons for voids, adjustments, etc.

Column	Type	Req.	Notes
category	text	yes	void, refund, discount, price_override, stock_adjustment, write_off, transfer_variance, till_variance
code	text	yes	
label	text	yes	
requires_note	boolean	yes	default false
is_active	boolean	yes	default true

Constraints: unique (tenant_id, category, code)

kernel.approval_rule The approval matrix.

Column	Type	Req.	Notes
company_id	uuid	yes	→ kernel.company
doc_type	text	yes	
min_amount	money	yes	default 0
max_amount	money		Null = no upper limit
step	smallint	yes	default 1
approver_role_id	uuid	yes	→ kernel.role
forbid_creator	boolean	yes	default true; Separation of duties
is_active	boolean	yes	default true

kernel.approval_request An approval waiting for or given a decision.

Column	Type	Req.	Notes
rule_id	uuid		→ kernel.approval_rule
doc_type	text	yes	
doc_id	uuid	yes	
step	smallint	yes	default 1
status	text	yes	one of: pending, approved, rejected, expired, cancelled; default 'pending'
requested_by	uuid		→ kernel.app_user
decided_by	uuid		→ kernel.app_user
decided_at	timestamp		
channel	text		one of: web, pos, whatsapp, api
note	text		
expires_at	timestamp		

kernel.workflow_rule Automation rules: trigger, conditions, actions.

Column	Type	Req.	Notes
name	text	yes	
version	integer	yes	default 1
trigger	jsonb	yes	Event type or schedule
conditions	jsonb	yes	default '[]'
actions	jsonb	yes	
is_active	boolean	yes	default true

kernel.workflow_run Each execution of a rule. line table

Column	Type	Req.	Notes
rule_id	uuid	yes	→ kernel.workflow_rule
event_id	bigint		
status	text	yes	one of: succeeded, failed, skipped
result	jsonb		

kernel.event_outbox Domain events written in the same transaction as the change. system table

Column	Type	Req.	Notes
id	bigint	yes	primary key
tenant_id	uuid	yes	→ kernel.tenant
event_type	text	yes	e.g. sale.posted
aggregate_type	text	yes	
aggregate_id	uuid	yes	
payload	jsonb	yes	
created_at	timestamp	yes	default now()
published_at	timestamp		
attempts	integer	yes	default 0

kernel.audit_log Append-only audit trail with hash chain. system table, immutable

Column	Type	Req.	Notes
id	bigint	yes	primary key
tenant_id	uuid	yes	→ kernel.tenant
occurred_at	timestamp	yes	default now()
user_id	uuid		→ kernel.app_user
channel	text	yes	one of: web, pos, api, whatsapp, system, import
device_id	uuid		
ip_address	inet		
action	text	yes	e.g. create, update, approve, post, void, login
entity_type	text	yes	
entity_id	uuid		
changes	jsonb		Previous and new values
prev_hash	text		
hash	text	yes	

kernel.idempotency_key Prevents duplicate creates on retry. system table

Column	Type	Req.	Notes
tenant_id	uuid	yes	→ kernel.tenant
key	text	yes	
request_hash	text	yes	
response	jsonb		
created_at	timestamp	yes	default now()

Constraints: primary key (tenant_id, key)

kernel.file Uploaded or generated files.

Column	Type	Req.	Notes
storage_key	text	yes	unique
filename	text	yes	
content_type	text	yes	
size_bytes	bigint	yes	

`kernel.notification` Messages sent to users.

Column	Type	Req.	Notes
user_id	uuid		→ kernel.app_user
channel	text	yes	one of: whatsapp, sms, email, in_app
template	text	yes	
payload	jsonb	yes	default '{}'
status	text	yes	one of: queued, sent, delivered, read, failed; default 'queued'
provider_message_id	text		
sent_at	timestamp		

`kernel.integration_credential` Encrypted credentials for Daraja, banks, eTIMS, WhatsApp.

Column	Type	Req.	Notes
company_id	uuid	yes	→ kernel.company
branch_id	uuid		→ kernel.branch
provider	text	yes	one of: daraja, bank, etims, whatsapp, email, sms
label	text	yes	
secret_ciphertext	bytea	yes	Encrypted; never logged
config	jsonb	yes	default '{}'; Non-secret settings, e.g. shortcode
status	text	yes	one of: active, disabled; default 'active'

`kernel.integration_log` Calls to and from external systems. line table

Column	Type	Req.	Notes
provider	text	yes	
direction	text	yes	one of: inbound, outbound
operation	text	yes	
status	text	yes	one of: ok, error, retrying
related_type	text		
related_id	uuid		
error	text		
duration_ms	integer		

`kernel.etims_device` eTIMS registration per branch.

Column	Type	Req.	Notes
company_id	uuid	yes	→ kernel.company
branch_id	uuid	yes	→ kernel.branch
device_serial	text	yes	
registration	jsonb	yes	default '{}'
status	text	yes	one of: pending, active, inactive; default 'active'

Constraints: unique (branch_id, device_serial)

kernel.etims_submission Submission of a sales or credit document to KRA eTIMS.

Column	Type	Req.	Notes
etims_device_id	uuid	yes	→ kernel.etims_device
doc_type	text	yes	one of: sale, invoice, credit_note, purchase
doc_id	uuid	yes	
status	text	yes	one of: pending, submitted, failed; default 'pending'
attempts	integer	yes	default 0
kra_invoice_no	text		
signature	text		
qr_data	text		
submitted_at	timestamp		
last_error	text		

Constraints: unique (doc_type, doc_id)

Schema `products` · Products and Pricing

One catalogue used by sales, purchasing and inventory. Requires: kernel. Migration: `010_products`.

products.category Product category tree.

Column	Type	Req.	Notes
parent_id	uuid		→ products.category
name	text	yes	

products.uom Units of measure.

Column	Type	Req.	Notes
code	text	yes	e.g. PCS, CTN, KG
name	text	yes	

Constraints: unique (tenant_id, code)

products.tax_code VAT class and eTIMS tax type.

Column	Type	Req.	Notes
code	text	yes	
name	text	yes	
vat_rate	numeric(5,2)	yes	Percent
etims_tax_type	text	yes	
is_active	boolean	yes	default true

Constraints: unique (tenant_id, code)

products.product A product or service.

Column	Type	Req.	Notes
company_id	uuid	yes	→ kernel.company
sku	text	yes	
name	text	yes	
category_id	uuid		→ products.category
product_type	text	yes	one of: stock, non_stock, service; default 'stock'
base_uom_id	uuid	yes	→ products.uom
tax_code_id	uuid	yes	→ products.tax_code
etims_item_class	text		KRA item classification code
is_batch_tracked	boolean	yes	default false
reorder_level	quantity		
is_active	boolean	yes	default true

Constraints: unique (company_id, sku)

products.product_uom Other units a product is sold or bought in. line table

Column	Type	Req.	Notes
product_id	uuid	yes	→ products.product
uom_id	uuid	yes	→ products.uom
factor_to_base	rate	yes	must be > 0; e.g. 1 carton = 24 pieces

Constraints: unique (product_id, uom_id)

products.barcode Barcodes per product and unit. line table

Column	Type	Req.	Notes
product_id	uuid	yes	→ products.product
uom_id	uuid		→ products.uom
code	text	yes	

Constraints: unique (tenant_id, code)

products.price_list A set of prices, optionally for a branch or customer group.

Column	Type	Req.	Notes
company_id	uuid	yes	→ kernel.company
name	text	yes	
branch_id	uuid		→ kernel.branch; Null = all branches
customer_group	text		
currency	char(3)	yes	default 'KES'
is_default	boolean	yes	default false

products.price_list_item Dated price for a product (versioned).

Column	Type	Req.	Notes
price_list_id	uuid	yes	→ products.price_list
product_id	uuid	yes	→ products.product
uom_id	uuid	yes	→ products.uom
price	money	yes	must be >= 0; Tax-inclusive selling price
valid_from	date	yes	
valid_to	date		
approved_by	uuid		→ kernel.app_user

Schema `crm` · CRM and Customers

Leads, customers, contacts and credit. Requires: kernel. Migration: `020_crm`.

crm.customer A customer.

Column	Type	Req.	Notes
company_id	uuid	yes	→ kernel.company
code	text	yes	
name	text	yes	
kind	text	yes	one of: individual, business; default 'business'
phone	text		
email	text		
kra_pin	text		
customer_group	text		
credit_limit	money	yes	default 0; Current value; history in customer_credit_version
payment_terms_days	integer	yes	default 0
owner_user_id	uuid		→ kernel.app_user; Account manager
status	text	yes	one of: active, on_hold, inactive; default 'active'

Constraints: unique (company_id, code)

crm.customer_contact People at a business customer. line table

Column	Type	Req.	Notes
customer_id	uuid	yes	→ crm.customer
name	text	yes	
phone	text		
email	text		
role	text		

`crm.customer_credit_version` History of credit limit and terms.

Column	Type	Req.	Notes
customer_id	uuid	yes	→ crm.customer
credit_limit	money	yes	
payment_terms_days	integer	yes	
valid_from	timestamp	yes	
reason	text		
approved_by	uuid		→ kernel.app_user

`crm.lead_source` Where leads come from.

Column	Type	Req.	Notes
name	text	yes	

Constraints: unique (tenant_id, name)

`crm.lead` A prospective customer.

Column	Type	Req.	Notes
company_id	uuid	yes	→ kernel.company
name	text	yes	
business_name	text		
phone	text		
email	text		
source_id	uuid		→ crm.lead_source
status	text	yes	one of: new, contacted, qualified, converted, lost; default 'new'
owner_user_id	uuid		→ kernel.app_user
converted_customer_id	uuid		→ crm.customer

`crm.activity` Calls, visits and follow-ups.

Column	Type	Req.	Notes
related_type	text	yes	one of: lead, customer
related_id	uuid	yes	
kind	text	yes	one of: call, visit, message, meeting, task
due_at	timestamp		
done_at	timestamp		
owner_user_id	uuid		→ kernel.app_user
note	text		

Schema `inventory` · Inventory and Branches

Stock ledger, balances, costing, transfers and counts. Requires: kernel, products. Migration: `030_inventory`.

`inventory.location` A place stock is held.

Column	Type	Req.	Notes
company_id	uuid	yes	→ kernel.company
branch_id	uuid		→ kernel.branch; Null for in-transit or virtual locations
code	text	yes	
name	text	yes	
kind	text	yes	one of: store, warehouse, transit, virtual; default 'store'
is_active	boolean	yes	default true

Constraints: unique (company_id, code)

`inventory.batch` Lot with expiry for batch-tracked products.

Column	Type	Req.	Notes
product_id	uuid	yes	→ products.product
batch_no	text	yes	
expiry_date	date		

Constraints: unique (product_id, batch_no)

`inventory.stock_movement` The stock ledger. Every quantity change; never updated. immutable

Column	Type	Req.	Notes
company_id	uuid	yes	→ kernel.company
branch_id	uuid		→ kernel.branch
location_id	uuid	yes	→ inventory.location
product_id	uuid	yes	→ products.product
batch_id	uuid		→ inventory.batch
quantity	quantity	yes	must be <> 0; + in, - out, in base unit
unit_cost	rate	yes	
value	money	yes	
movement_type	text	yes	one of: sale, sale_return, receipt, purchase_return, transfer_out, transfer_in, adjustment, count_variance, write_off, opening
source_type	text	yes	
source_id	uuid	yes	
source_line_id	uuid		
reason_code_id	uuid		→ kernel.reason_code
business_date	date	yes	
approved_by	uuid		→ kernel.app_user

Constraints: check (movement_type not in ('adjustment','count_variance','write_off') or reason_code_id is not null)

`inventory.stock_balance` Current quantity per location, product and batch (derived). line table

Column	Type	Req.	Notes
location_id	uuid	yes	→ inventory.location
product_id	uuid	yes	→ products.product
batch_id	uuid		→ inventory.batch
quantity	quantity	yes	default 0
updated_at	timestamp	yes	default now()

Constraints: unique nulls not distinct (location_id, product_id, batch_id)

`inventory.product_cost` Weighted average cost per company and product (derived). line table

Column	Type	Req.	Notes
company_id	uuid	yes	→ kernel.company
product_id	uuid	yes	→ products.product
quantity	quantity	yes	default 0
value	money	yes	default 0
average_cost	rate	yes	default 0
updated_at	timestamp	yes	default now()

Constraints: unique (company_id, product_id)

`inventory.transfer` Stock moved between locations.

Column	Type	Req.	Notes
company_id	uuid	yes	→ kernel.company
number	text	yes	
from_location_id	uuid	yes	→ inventory.location
to_location_id	uuid	yes	→ inventory.location
status	text	yes	one of: draft, pending_approval, dispatched, received, cancelled; default 'draft'
dispatched_at	timestamp		
dispatched_by	uuid		→ kernel.app_user
received_at	timestamp		
received_by	uuid		→ kernel.app_user

Constraints: unique (company_id, number); check (from_location_id <> to_location_id)

`inventory.transfer_line` Items in a transfer. line table

Column	Type	Req.	Notes
transfer_id	uuid	yes	→ inventory.transfer
product_id	uuid	yes	→ products.product
batch_id	uuid		→ inventory.batch
qty_sent	quantity	yes	must be > 0
qty_received	quantity		
variance_reason_code_id	uuid		→ kernel.reason_code

`inventory.stock_adjustment` A manual change to stock with a reason.

Column	Type	Req.	Notes
company_id	uuid	yes	→ kernel.company
number	text	yes	
location_id	uuid	yes	→ inventory.location
reason_code_id	uuid	yes	→ kernel.reason_code
note	text		
status	text	yes	one of: draft, pending_approval, posted, rejected; default 'draft'
approved_by	uuid		→ kernel.app_user
posted_at	timestamp		

Constraints: unique (company_id, number)

`inventory.stock_adjustment_line` Items in an adjustment. line table

Column	Type	Req.	Notes
adjustment_id	uuid	yes	→ inventory.stock_adjustment
product_id	uuid	yes	→ products.product
batch_id	uuid		→ inventory.batch
qty_change	quantity	yes	must be <> 0

`inventory.stock_count` A physical count.

Column	Type	Req.	Notes
company_id	uuid	yes	→ kernel.company
number	text	yes	
location_id	uuid	yes	→ inventory.location
status	text	yes	one of: counting, review, pending_approval, posted, cancelled; default 'counting'
snapshot_at	timestamp	yes	
approved_by	uuid		→ kernel.app_user
posted_at	timestamp		

Constraints: unique (company_id, number)

`inventory.stock_count_line` Expected and counted quantities. line table

Column	Type	Req.	Notes
count_id	uuid	yes	→ inventory.stock_count
product_id	uuid	yes	→ products.product
batch_id	uuid		→ inventory.batch
expected_qty	quantity	yes	
counted_qty	quantity		
variance_qty	quantity		
reason_code_id	uuid		→ kernel.reason_code

Schema `payments` · Payments and Reconciliation

Money in and out, M-Pesa and bank data, matching, daily close. Requires: kernel. Migration: `040_payments`.

`payments.payment_account` A till, Paybill, bank account or cash account.

Column	Type	Req.	Notes
company_id	uuid	yes	→ kernel.company
branch_id	uuid		→ kernel.branch
kind	text	yes	one of: cash, mpesa_till, mpesa_paybill, bank, card
name	text	yes	
shortcode	text		M-Pesa Till or Paybill number
bank_account_masked	text		
credential_id	uuid		→ kernel.integration_credential
ledger_account_id	uuid		Soft reference to finance.account
is_active	boolean	yes	default true

`payments.external_transaction` A raw line from M-Pesa or a bank, normalised.

Column	Type	Req.	Notes
payment_account_id	uuid	yes	→ payments.payment_account
source	text	yes	one of: daraja_c2b, daraja_stk, daraja_pull, mpesa_statement, bank_api, bank_statement
external_ref	text	yes	M-Pesa receipt or bank reference
txn_time	timestamp	yes	
direction	text	yes	one of: in, out
amount	money	yes	must be > 0
fee	money	yes	default 0
bill_ref	text		Paybill account number entered by payer
counterparty_name	text		
counterparty_phone_masked	text		
raw	jsonb	yes	
match_status	text	yes	one of: unmatched, proposed, matched, ignored; default 'unmatched'

Constraints: unique (payment_account_id, external_ref)

payments.payment_intent A payment request, e.g. an STK push.

Column	Type	Req.	Notes
payment_account_id	uuid	yes	→ payments.payment_account
kind	text	yes	one of: stk_push, b2c
amount	money	yes	must be > 0
phone_masked	text	yes	
provider_request_id	text		unique; e.g. CheckoutRequestID
status	text	yes	one of: pending, succeeded, failed, cancelled, timeout; default 'pending'
result_code	text		
external_transaction_id	uuid		→ payments.external_transaction
doc_type	text		
doc_id	uuid		

payments.payment Money received or paid, linked to a party.

Column	Type	Req.	Notes
company_id	uuid	yes	→ kernel.company
branch_id	uuid		→ kernel.branch
number	text	yes	
direction	text	yes	one of: in, out
method	text	yes	one of: cash, mpesa, bank, card, cheque, other
payment_account_id	uuid	yes	→ payments.payment_account
amount	money	yes	must be > 0
business_date	date	yes	
party_type	text		one of: customer, supplier, employee, other
party_id	uuid		Soft reference to the party's module
external_ref	text		
status	text	yes	one of: draft, posted, reversed; default 'posted'

Constraints: unique (company_id, number)

payments.payment_allocation What a payment pays for. line table

Column	Type	Req.	Notes
payment_id	uuid	yes	→ payments.payment
doc_type	text	yes	sale, invoice, supplier_bill, payslip
doc_id	uuid	yes	
amount	money	yes	must be > 0

`payments.match` Link between an external transaction and a payment.

Column	Type	Req.	Notes
external_transaction_id	uuid	yes	→ payments.external_transaction
payment_id	uuid	yes	→ payments.payment
amount	money	yes	
stage	text	yes	one of: exact_reference, account_number, rule, manual
rule_code	text		
confidence	numeric(5,4)		
status	text	yes	one of: proposed, applied, undone; default 'applied'
undone_by	uuid		→ kernel.app_user
undone_at	timestamp		

`payments.daily_close` Branch close for a business date.

Column	Type	Req.	Notes
company_id	uuid	yes	→ kernel.company
branch_id	uuid	yes	→ kernel.branch
business_date	date	yes	
status	text	yes	one of: open, submitted, locked, reopened; default 'open'
totals	jsonb	yes	default '{}'; Expected and actual per method
variance	money	yes	default 0
submitted_by	uuid		→ kernel.app_user
submitted_at	timestamp		
locked_at	timestamp		

Constraints: unique (branch_id, business_date)

`payments.recon_exception` An item needing review.

Column	Type	Req.	Notes
branch_id	uuid		→ kernel.branch
kind	text	yes	unmatched_receipt, unpaid_sale, close_variance, duplicate_ref, ...
ref_type	text	yes	
ref_id	uuid	yes	
amount	money		
status	text	yes	one of: open, explained, resolved, dismissed; default 'open'
assigned_to	uuid		→ kernel.app_user
resolution_note	text		
resolved_by	uuid		→ kernel.app_user
resolved_at	timestamp		

Schema `pos` · Point of Sale and Sales

POS devices, till sessions, sales, quotations, orders, invoices and returns. Requires: kernel, products, inventory, payments. Migration: `050_pos`.

`pos.pos_device` A registered POS device.

Column	Type	Req.	Notes
branch_id	uuid	yes	→ kernel.branch
name	text	yes	
device_key_hash	text	yes	unique
status	text	yes	one of: active, blocked; default 'active'
last_sync_at	timestamp		

`pos.till_session` A cashier's session with a cash float.

Column	Type	Req.	Notes
branch_id	uuid	yes	→ kernel.branch
device_id	uuid	yes	→ pos.pos_device
cash_account_id	uuid	yes	→ payments.payment_account
cashier_id	uuid	yes	→ kernel.app_user
business_date	date	yes	
opened_at	timestamp	yes	
opening_float	money	yes	default 0
closed_at	timestamp		
expected_cash	money		
counted_cash	money		
variance	money		
variance_reason_code_id	uuid		→ kernel.reason_code
status	text	yes	one of: open, closed; default 'open'

pos.sale A POS or counter sale.

Column	Type	Req.	Notes
company_id	uuid	yes	→ kernel.company
branch_id	uuid	yes	→ kernel.branch
number	text	yes	Per-device series
device_id	uuid		→ pos.pos_device
local_id	uuid	yes	Device-generated ID for offline sync
till_session_id	uuid		→ pos.till_session
customer_id	uuid		Soft reference to crm.customer
cashier_id	uuid	yes	→ kernel.app_user
business_date	date	yes	
sold_at	timestamp	yes	
subtotal	money	yes	
discount_total	money	yes	default 0
tax_total	money	yes	
total	money	yes	
paid_total	money	yes	default 0
status	text	yes	one of: posted, voided; default 'posted'
voided_by	uuid		→ kernel.app_user
voided_at	timestamp		
void_reason_code_id	uuid		→ kernel.reason_code
synced_offline	boolean	yes	default false

Constraints: unique (company_id, number); unique (tenant_id, local_id); check (status <> 'voided' or void_reason_code_id is not null)

pos.sale_line Items in a sale. line table

Column	Type	Req.	Notes
sale_id	uuid	yes	→ pos.sale
line_no	smallint	yes	
product_id	uuid	yes	→ products.product
uom_id	uuid	yes	→ products.uom
batch_id	uuid		→ inventory.batch
quantity	quantity	yes	must be > 0
list_price	money	yes	
unit_price	money	yes	
discount_amount	money	yes	default 0
price_reason_code_id	uuid		→ kernel.reason_code; Required when price or discount exceeds policy
tax_code_id	uuid	yes	→ products.tax_code
tax_amount	money	yes	
line_total	money	yes	
cost_amount	money		Set at posting from average cost

Constraints: unique (sale_id, line_no)

pos.sale_payment How a sale was paid. line table

Column	Type	Req.	Notes
sale_id	uuid	yes	→ pos.sale
method	text	yes	one of: cash, mpesa, card, credit, other
payment_account_id	uuid		→ payments.payment_account
amount	money	yes	
external_ref	text		M-Pesa receipt code if known
payment_id	uuid		→ payments.payment

pos.sales_document Quotation or sales order.

Column	Type	Req.	Notes
company_id	uuid	yes	→ kernel.company
branch_id	uuid		→ kernel.branch
doc_kind	text	yes	one of: quotation, order
number	text	yes	
customer_id	uuid		Soft reference to crm.customer
doc_date	date	yes	
valid_until	date		
source_quotation_id	uuid		→ pos.sales_document
total	money	yes	default 0
status	text	yes	one of: draft, pending_approval, sent, confirmed, fulfilled, cancelled, expired; default 'draft'
approved_by	uuid		→ kernel.app_user

Constraints: unique (company_id, number)

pos.sales_document_line Lines of a quotation or order. line table

Column	Type	Req.	Notes
document_id	uuid	yes	→ pos.sales_document
product_id	uuid	yes	→ products.product
uom_id	uuid	yes	→ products.uom
quantity	quantity	yes	must be > 0
unit_price	money	yes	
discount_amount	money	yes	default 0
tax_code_id	uuid	yes	→ products.tax_code
line_total	money	yes	
qty_delivered	quantity	yes	default 0

`pos.sales_invoice` A tax invoice to a customer (credit or B2B).

Column	Type	Req.	Notes
company_id	uuid	yes	→ kernel.company
branch_id	uuid		→ kernel.branch
number	text	yes	
customer_id	uuid	yes	Soft reference to crm.customer
sale_id	uuid		→ pos.sale
sales_order_id	uuid		→ pos.sales_document
issue_date	date	yes	
due_date	date	yes	
subtotal	money	yes	
tax_total	money	yes	
total	money	yes	
amount_paid	money	yes	default 0
status	text	yes	one of: draft, issued, part_paid, paid, cancelled; default 'issued'

Constraints: unique (company_id, number)

`pos.sales_invoice_line` Lines of an invoice. line table

Column	Type	Req.	Notes
invoice_id	uuid	yes	→ pos.sales_invoice
product_id	uuid	yes	→ products.product
uom_id	uuid	yes	→ products.uom
quantity	quantity	yes	must be > 0
unit_price	money	yes	
discount_amount	money	yes	default 0
tax_code_id	uuid	yes	→ products.tax_code
tax_amount	money	yes	
line_total	money	yes	

`pos.sales_return` Return of goods; produces a credit note.

Column	Type	Req.	Notes
company_id	uuid	yes	→ kernel.company
branch_id	uuid		→ kernel.branch
number	text	yes	
original_sale_id	uuid		→ pos.sale
original_invoice_id	uuid		→ pos.sales_invoice
reason_code_id	uuid	yes	→ kernel.reason_code
refund_method	text	yes	one of: cash, mpesa, credit_note, exchange
total	money	yes	
restock_location_id	uuid		→ inventory.location
status	text	yes	one of: draft, pending_approval, posted, rejected; default 'draft'
approved_by	uuid		→ kernel.app_user

Constraints: unique (company_id, number); check (original_sale_id is not null or original_invoice_id is not null)

`pos.sales_return_line` Items returned. line table

Column	Type	Req.	Notes
return_id	uuid	yes	→ pos.sales_return
product_id	uuid	yes	→ products.product
batch_id	uuid		→ inventory.batch
quantity	quantity	yes	must be > 0
unit_price	money	yes	
tax_amount	money	yes	
line_total	money	yes	

Schema `purchasing` · Purchasing and Suppliers

Suppliers, requests, orders, receipts and bills. Requires: kernel, products. Migration: `060_purchasing`.

`purchasing.supplier` A supplier.

Column	Type	Req.	Notes
company_id	uuid	yes	→ kernel.company
code	text	yes	
name	text	yes	
kra_pin	text		
phone	text		
email	text		
payment_terms_days	integer	yes	default 0
status	text	yes	one of: active, on_hold, inactive; default 'active'

Constraints: unique (company_id, code)

`purchasing.supplier_bank_account` Supplier payment details (versioned; changes approved).

Column	Type	Req.	Notes
supplier_id	uuid	yes	→ purchasing.supplier
method	text	yes	one of: bank, mpesa
bank_name	text		
account_name	text		
account_no_ciphertext	bytea		Encrypted
account_no_masked	text		
valid_from	timestamp	yes	
valid_to	timestamp		
approved_by	uuid		→ kernel.app_user
confirmed_by	uuid		→ kernel.app_user; Person who confirmed directly with the supplier
confirmation_method	text		

`purchasing.purchase_request` Internal request to buy.

Column	Type	Req.	Notes
company_id	uuid	yes	→ kernel.company
branch_id	uuid		→ kernel.branch
number	text	yes	
needed_by	date		
status	text	yes	one of: draft, pending_approval, approved, rejected, ordered; default 'draft'
approved_by	uuid		→ kernel.app_user

Constraints: unique (company_id, number)

`purchasing.purchase_request_line` Items requested. line table

Column	Type	Req.	Notes
request_id	uuid	yes	→ purchasing.purchase_request
product_id	uuid	yes	→ products.product
quantity	quantity	yes	must be > 0
note	text		

`purchasing.purchase_order` Order to a supplier.

Column	Type	Req.	Notes
company_id	uuid	yes	→ kernel.company
branch_id	uuid		→ kernel.branch
number	text	yes	
supplier_id	uuid	yes	→ purchasing.supplier
request_id	uuid		→ purchasing.purchase_request
deliver_to_location_id	uuid		Soft reference to inventory.location
order_date	date	yes	
expected_date	date		
subtotal	money	yes	default 0
tax_total	money	yes	default 0
total	money	yes	default 0
status	text	yes	one of: draft, pending_approval, approved, sent, part_received, received, closed, cancelled; default 'draft'
approved_by	uuid		→ kernel.app_user

Constraints: unique (company_id, number)

`purchasing.purchase_order_line` Items ordered. line table

Column	Type	Req.	Notes
order_id	uuid	yes	→ purchasing.purchase_order
product_id	uuid	yes	→ products.product
uom_id	uuid	yes	→ products.uom
quantity	quantity	yes	must be > 0
unit_cost	rate	yes	
tax_code_id	uuid	yes	→ products.tax_code
qty_received	quantity	yes	default 0
qty_billed	quantity	yes	default 0

`purchasing.goods_receipt` Goods received against an order.

Column	Type	Req.	Notes
company_id	uuid	yes	→ kernel.company
branch_id	uuid		→ kernel.branch
number	text	yes	
order_id	uuid		→ purchasing.purchase_order
supplier_id	uuid	yes	→ purchasing.supplier
location_id	uuid		Soft reference to inventory.location
received_at	timestamp	yes	
delivery_note_no	text		
status	text	yes	one of: draft, posted, reversed; default 'draft'

Constraints: unique (company_id, number)

`purchasing.goods_receipt_line` Items received. line table

Column	Type	Req.	Notes
receipt_id	uuid	yes	→ purchasing.goods_receipt
order_line_id	uuid		→ purchasing.purchase_order_line
product_id	uuid	yes	→ products.product
batch_no	text		
expiry_date	date		
quantity	quantity	yes	must be > 0
unit_cost	rate	yes	
landed_cost	money	yes	default 0

`purchasing.supplier_bill` Supplier invoice.

Column	Type	Req.	Notes
company_id	uuid	yes	→ kernel.company
branch_id	uuid		→ kernel.branch
number	text	yes	
supplier_id	uuid	yes	→ purchasing.supplier
supplier_invoice_no	text	yes	
bill_date	date	yes	
due_date	date	yes	
subtotal	money	yes	
tax_total	money	yes	
total	money	yes	
amount_paid	money	yes	default 0
match_status	text	yes	one of: pending, matched, exception, not_required; default 'pending'
etims_ref	text		
status	text	yes	one of: draft, pending_approval, posted, part_paid, paid, cancelled; default 'draft'
approved_by	uuid		→ kernel.app_user

Constraints: unique (company_id, number); unique (supplier_id, supplier_invoice_no)

`purchasing.supplier_bill_line` Lines of a supplier bill. line table

Column	Type	Req.	Notes
bill_id	uuid	yes	→ purchasing.supplier_bill
order_line_id	uuid		→ purchasing.purchase_order_line
receipt_line_id	uuid		→ purchasing.goods_receipt_line
product_id	uuid		→ products.product
expense_account_id	uuid		Soft reference to finance.account, for non-stock lines
description	text		
quantity	quantity	yes	
unit_cost	rate	yes	
tax_code_id	uuid		→ products.tax_code
tax_amount	money	yes	default 0
line_total	money	yes	

`purchasing.supplier_price` Cost history per supplier and product. line table

Column	Type	Req.	Notes
supplier_id	uuid	yes	→ purchasing.supplier
product_id	uuid	yes	→ products.product
unit_cost	rate	yes	
effective_date	date	yes	
source_type	text	yes	
source_id	uuid	yes	

Schema `finance` · Finance and Accounting

Chart of accounts, journals, periods, expenses and bank reconciliation. Requires: kernel. Migration: `070_finance`.

`finance.account` Chart of accounts.

Column	Type	Req.	Notes
company_id	uuid	yes	→ kernel.company
code	text	yes	
name	text	yes	
type	text	yes	one of: asset, liability, equity, income, expense
subtype	text		cash, bank, mpesa_clearing, receivable, payable, inventory, vat_input, vat_output, cogs, ...
parent_id	uuid		→ finance.account
is_control	boolean	yes	default false; Sub-ledger control account; no manual posting
is_active	boolean	yes	default true

Constraints: unique (company_id, code)

`finance.period` Accounting periods.

Column	Type	Req.	Notes
company_id	uuid	yes	→ kernel.company
start_date	date	yes	
end_date	date	yes	
status	text	yes	one of: open, soft_closed, locked; default 'open'
closed_by	uuid		→ kernel.app_user
closed_at	timestamp		

Constraints: unique (company_id, start_date); check (end_date >= start_date)

`finance.posting_rule` Maps document lines to accounts.

Column	Type	Req.	Notes
company_id	uuid	yes	→ kernel.company
doc_type	text	yes	e.g. sale, goods_receipt, payroll
line_role	text	yes	e.g. revenue, vat_output, cogs, inventory
account_id	uuid	yes	→ finance.account
side	text	yes	one of: debit, credit
conditions	jsonb	yes	default '{}'; e.g. payment method, product category
priority	integer	yes	default 100

`finance.journal` Journal books.

Column	Type	Req.	Notes
company_id	uuid	yes	→ kernel.company
code	text	yes	SAL, PUR, CSH, PAY, GEN
name	text	yes	

Constraints: unique (company_id, code)

`finance.journal_entry` A balanced accounting entry. immutable

Column	Type	Req.	Notes
company_id	uuid	yes	→ kernel.company
journal_id	uuid	yes	→ finance.journal
period_id	uuid	yes	→ finance.period
number	text	yes	
entry_date	date	yes	
source_type	text		Originating document type; null for manual
source_id	uuid		
description	text		
is_manual	boolean	yes	default false
reversal_of_id	uuid		→ finance.journal_entry
approved_by	uuid		→ kernel.app_user
posted_at	timestamp	yes	default now()

Constraints: unique (company_id, number)

`finance.journal_line` Debit or credit line. Entries must balance. line table, immutable

Column	Type	Req.	Notes
entry_id	uuid	yes	→ finance.journal_entry
company_id	uuid	yes	→ kernel.company
branch_id	uuid		→ kernel.branch
account_id	uuid	yes	→ finance.account
debit	money	yes	default 0; must be >= 0
credit	money	yes	default 0; must be >= 0
party_type	text		
party_id	uuid		
description	text		

Constraints: check ((debit = 0) <> (credit = 0))

finance.expense An expense or petty cash claim.

Column	Type	Req.	Notes
company_id	uuid	yes	→ kernel.company
branch_id	uuid		→ kernel.branch
number	text	yes	
expense_date	date	yes	
payee	text		
account_id	uuid	yes	→ finance.account
amount	money	yes	must be > 0
tax_amount	money	yes	default 0
paid_from_account_id	uuid		Soft reference to payments.payment_account
receipt_file_id	uuid		→ kernel.file
status	text	yes	one of: draft, pending_approval, approved, posted, rejected; default 'draft'
approved_by	uuid		→ kernel.app_user

Constraints: unique (company_id, number)

finance.bank_reconciliation Reconciliation of a bank or M-Pesa account to a statement.

Column	Type	Req.	Notes
company_id	uuid	yes	→ kernel.company
payment_account_id	uuid	yes	Soft reference to payments.payment_account
statement_date	date	yes	
statement_balance	money	yes	
book_balance	money	yes	
difference	money	yes	
status	text	yes	one of: in_progress, completed; default 'in_progress'
completed_by	uuid		→ kernel.app_user
completed_at	timestamp		

Schema `people` · People, HR and Payroll

Employees, attendance, leave and payroll. Requires: kernel. Migration: `080_people`.

people.department Departments.

Column	Type	Req.	Notes
company_id	uuid	yes	→ kernel.company
name	text	yes	

Constraints: unique (company_id, name)

people.employee An employee.

Column	Type	Req.	Notes
company_id	uuid	yes	→ kernel.company
branch_id	uuid		→ kernel.branch
department_id	uuid		→ people.department
user_id	uuid		→ kernel.app_user; Link to the POS or system user
employee_no	text	yes	
full_name	text	yes	
national_id_ciphertext	bytea		Encrypted
kra_pin	text		
nssf_no	text		
shif_no	text		
phone	text		
email	text		
hire_date	date	yes	
termination_date	date		
status	text	yes	one of: active, on_leave, terminated; default 'active'

Constraints: unique (company_id, employee_no)

people.pay_component Earnings, deductions and benefits.

Column	Type	Req.	Notes
company_id	uuid	yes	→ kernel.company
code	text	yes	
name	text	yes	
kind	text	yes	one of: earning, deduction, benefit, employer_contribution
is_taxable	boolean	yes	default true
is_statutory	boolean	yes	default false
statutory_code	text		Links to statutory_rate_table.code

Constraints: unique (company_id, code)

people.employment_terms Basic pay and terms (versioned).

Column	Type	Req.	Notes
employee_id	uuid	yes	→ people.employee
basic_pay	money	yes	must be >= 0
pay_frequency	text	yes	one of: monthly; default 'monthly'
valid_from	date	yes	
valid_to	date		
approved_by	uuid		→ kernel.app_user

people.employee_pay_item Recurring or one-off pay items.

Column	Type	Req.	Notes
employee_id	uuid	yes	→ people.employee
component_id	uuid	yes	→ people.pay_component
amount	money	yes	
recurring	boolean	yes	default true
valid_from	date	yes	
valid_to	date		

people.employee_bank_account Salary payment details (versioned; changes approved).

Column	Type	Req.	Notes
employee_id	uuid	yes	→ people.employee
method	text	yes	one of: bank, mpesa
bank_name	text		
account_no_ciphertext	bytea		
account_no_masked	text		
valid_from	timestamp	yes	
valid_to	timestamp		
approved_by	uuid		→ kernel.app_user

people.shift Shift definitions.

Column	Type	Req.	Notes
branch_id	uuid		→ kernel.branch
name	text	yes	
start_time	time	yes	
end_time	time	yes	

people.attendance Daily attendance.

Column	Type	Req.	Notes
employee_id	uuid	yes	→ people.employee
branch_id	uuid		→ kernel.branch
work_date	date	yes	
shift_id	uuid		→ people.shift
clock_in	timestamp		
clock_out	timestamp		
hours	numeric(6,2)		
overtime_hours	numeric(6,2)	yes	default 0
source	text	yes	one of: pos_login, device, manual

Constraints: unique (employee_id, work_date, shift_id)

`people.leave_type` Types of leave.

Column	Type	Req.	Notes
company_id	uuid	yes	→ kernel.company
name	text	yes	
days_per_year	numeric(6,2)		
is_paid	boolean	yes	default true

`people.leave_request` A request for leave.

Column	Type	Req.	Notes
employee_id	uuid	yes	→ people.employee
leave_type_id	uuid	yes	→ people.leave_type
start_date	date	yes	
end_date	date	yes	
days	numeric(6,2)	yes	
status	text	yes	one of: pending, approved, rejected, cancelled; default 'pending'
approved_by	uuid		→ kernel.app_user

Constraints: check (end_date >= start_date)

`people.statutory_rate_table` Effective-dated statutory rules (platform level). platform table, no tenant

Column	Type	Req.	Notes
id	uuid	yes	primary key
code	text	yes	one of: paye, shif, nssf, housing_levy, other
effective_from	date	yes	
effective_to	date		
definition	jsonb	yes	Bands, rates, limits, reliefs, calculation order
source_reference	text	yes	Law or notice the values come from
verified_by	text	yes	
verified_at	timestamp	yes	

Constraints: unique (code, effective_from)

`people.payroll_run` A monthly payroll run.

Column	Type	Req.	Notes
company_id	uuid	yes	→ kernel.company
period_start	date	yes	
period_end	date	yes	
pay_date	date	yes	
status	text	yes	one of: draft, calculated, pending_approval, approved, posted, paid, cancelled; default 'draft'
totals	jsonb	yes	default '{}'
prepared_by	uuid		→ kernel.app_user
approved_by	uuid		→ kernel.app_user
journal_entry_id	uuid		Soft reference to finance.journal_entry

Constraints: unique (company_id, period_start); check (approved_by is null or approved_by <> prepared_by)

people.payslip One employee's result in a run. line table

Column	Type	Req.	Notes
run_id	uuid	yes	→ people.payroll_run
employee_id	uuid	yes	→ people.employee
gross_pay	money	yes	
taxable_pay	money	yes	
total_deductions	money	yes	
net_pay	money	yes	
employer_cost	money	yes	
file_id	uuid		→ kernel.file
status	text	yes	one of: calculated, approved, paid; default 'calculated'

Constraints: unique (run_id, employee_id)

people.payslip_line Each earning and deduction on a payslip. line table

Column	Type	Req.	Notes
payslip_id	uuid	yes	→ people.payslip
component_code	text	yes	
kind	text	yes	one of: earning, deduction, benefit, employer_contribution
amount	money	yes	
rate_table_id	uuid		→ people.statutory_rate_table; Which rule version produced it

Schema command · Command Center

User preferences for dashboards and briefs. Findings live in Ziira Intelligence. Requires: kernel. Migration: 090_command.

command.brief_schedule When and how each user receives briefs.

Column	Type	Req.	Notes
user_id	uuid	yes	→ kernel.app_user
kind	text	yes	one of: daily, weekly
channel	text	yes	one of: whatsapp, email, in_app; default 'whatsapp'
send_time	time	yes	
weekday	smallint		
is_active	boolean	yes	default true

Constraints: unique (user_id, kind)

command.dashboard_widget Widgets on a user's Command Center. line table

Column	Type	Req.	Notes
user_id	uuid	yes	→ kernel.app_user
widget_code	text	yes	
position	smallint	yes	
settings	jsonb	yes	default '{}'

Schema `intel` · Ziira Intelligence store

Owned by the Ziira Intelligence service; shown here because Core reads findings from it. Requires: kernel. Migration: `100_intel`.

`intel.check_definition` Versioned checks (platform level). platform table, no tenant

Column	Type	Req.	Notes
code	text	yes	
version	integer	yes	
title	text	yes	
severity_default	text	yes	one of: info, low, medium, high, critical; default in
definition	jsonb	yes	
is_active	boolean	yes	default true
id	uuid	yes	primary key

Constraints: unique (code, version)

`intel.finding` A result of a check needing attention.

Column	Type	Req.	Notes
check_id	uuid	yes	→ intel.check_definition
company_id	uuid		→ kernel.company
branch_id	uuid		→ kernel.branch
entity_type	text		
entity_id	uuid		
period_start	date		
period_end	date		
value	numeric(20,4)		
expected_value	numeric(20,4)		
severity	text	yes	one of: info, low, medium, high, critical
evidence	jsonb	yes	Record IDs behind the finding
explanation	text		Plain-language text from the AI layer
status	text	yes	one of: new, sent, acknowledged, explained, resolved, dismissed; default 'new'
owner_user_id	uuid		→ kernel.app_user
first_seen_at	timestamp	yes	default now()
last_seen_at	timestamp	yes	default now()
resolved_at	timestamp		

`intel.finding_event` History of a finding: sent, explained, resolved. line table

Column	Type	Req.	Notes
finding_id	uuid	yes	→ intel.finding
event	text	yes	
user_id	uuid		→ kernel.app_user
channel	text		
note	text		

intel.baseline Normal ranges per tenant, used to detect unusual values. line table

Column	Type	Req.	Notes
metric	text	yes	e.g. void_rate
dimension	jsonb	yes	e.g. branch, cashier, weekday
mean	numeric(20,6)		
stddev	numeric(20,6)		
p95	numeric(20,6)		
sample_size	integer	yes	
computed_at	timestamp	yes	